

Tarea Programada: Editor, visualizador y comparador de archivos

Felipe Arce Castillo
Jose David Serrano Robles
Mariana Torres Valverde

Escuela de Ingeniería en Computación, Tecnológico de Costa Rica, San José, Costa Rica

josserrano@estudiantec.cr
wiyumiyu@estudiantec.cr
fearce@estudiantec.cr

Abstract – Este proyecto es un visor y editor de archivos de texto de bajo nivel implementado en lenguaje ensamblador x86-64 para Linux. La aplicación proporciona funcionalidades para abrir, leer, visualizar y editar archivos de texto directamente desde la línea de comandos. Además, incluye características para la visualización en formato hexadecimal del contenido del archivo y la comparación línea por línea entre dos archivos de texto.

Keywords - Ensamblador, ASM, editor de archivos.

optimizar el rendimiento del software al máximo. Este enfoque proporciona una valiosa experiencia práctica en programación a bajo nivel y en el manejo directo de hardware.

El desarrollo del proyecto se ha estructurado en varios pasos clave, que incluyen el diseño de diagramas de flujo y diagramas de bloques funcionales. Estos diagramas han sido fundamentales para planificar y visualizar la estructura y el funcionamiento del software antes de proceder a la codificación en ensamblador.

I. INTRODUCCIÓN

Este proyecto se centra en el desarrollo de un editor, visualizador y comparador de archivos. La solución ha sido implementada en ensamblador x8086 ASM, aprovechando la eficiencia y el control de bajo nivel que este lenguaje proporciona. El objetivo principal del trabajo es crear una herramienta que permita a los usuarios abrir, editar y comparar archivos de texto mediante una interfaz de comandos.

El editor de archivos desarrollado en este proyecto permite a los usuarios realizar operaciones básicas de edición como la inserción, eliminación y modificación de texto. Además, incorpora funcionalidades para visualizar el contenido de archivos y comparar dos archivos diferentes, resaltando las diferencias entre ellos.

La elección de ensamblador x8086 ASM como plataforma de desarrollo responde a la necesidad de comprender y manejar los aspectos más profundos de la arquitectura del procesador, así como a la intención de

Diagramas de Flujo: Estos diagramas detallan el proceso lógico de las operaciones principales del editor, visualizador y comparador de archivos. Incluyen la secuencia de comandos y decisiones que el programa debe seguir para ejecutar cada funcionalidad.

Diagramas de Bloques Funcionales: Estos diagramas representan los componentes principales del sistema y sus interacciones. Proveen una visión clara de cómo se integran los distintos módulos del software, tales como el manejo de archivos, la interfaz de usuario y los algoritmos de comparación de textos.

Al final del proyecto, se ha consolidado toda la información en un manual de usuario resumido. Este manual proporciona instrucciones claras y detalladas sobre cómo utilizar el editor, visualizador y comparador de archivos. Incluye ejemplos prácticos de comandos, descripciones de funcionalidades y guías paso a paso

para facilitar a los usuarios la comprensión y uso del software.

En pocas palabras, este proyecto ofrece una herramienta robusta y eficiente para la edición, visualización y comparación de archivos, desarrollada completamente en ensamblador x8086 ASM. A través de una planificación cuidadosa y una implementación detallada, se ha creado un software que no solo cumple con sus objetivos funcionales, sino que también sirve como un valioso recurso educativo en el ámbito de la programación a bajo nivel.

II. OBJETIVOS

Objetivo General:

Desarrollar un editor, visualizador y comparador de archivos en ensamblador x8086 ASM.

Objetivos Específicos:

- Diseñar e implementar las funciones básicas de edición, como inserción, eliminación y modificación de texto, en el editor de archivos.
- Desarrollar algoritmos para la visualización del contenido de archivos y la comparación de dos archivos diferentes, resaltando las diferencias entre ellos.
- Crear diagramas de flujo y diagramas de bloques funcionales para planificar y visualizar la estructura y el funcionamiento del software, facilitando la comprensión del proyecto y su posterior codificación en ensamblador.

III. ESQUEMAS DE DISEÑO Y ARQUITECTURA

3.1 Diagrama de bloques general

Este diagrama proporciona una visión general de la estructura del código, mostrando cómo se organiza y cómo interactúan las distintas partes del programa.

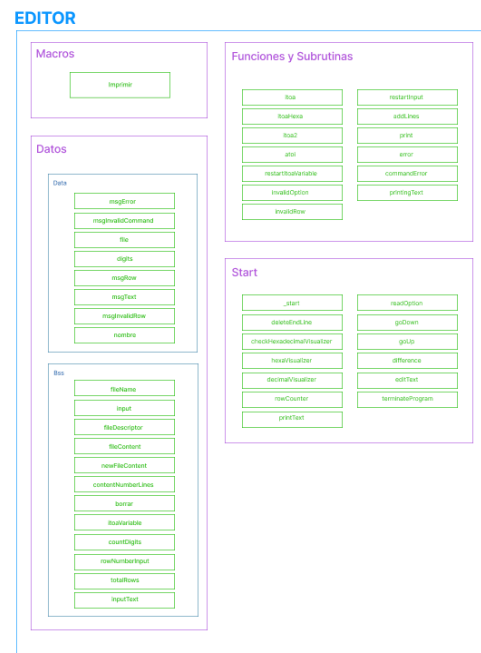


Figura 1: Diagrama de bloques para el editor

Dentro del programa principal se tienen 4 grandes bloques.

Macros: Es esta sección solo se tienen una y es una rutina de impresión; esta realiza una llamada al sistema para escribir datos en la salida estándar. Acepta dos parámetros: la dirección del buffer a imprimir y el número de bytes a imprimir.

Datos: Se dividen en dos, en la sección .data, se declaran datos estáticos o constantes que no cambian durante la ejecución del programa y en la sección la sección .bss, se declaran variables que serán inicializadas en tiempo de ejecución. Estas variables ocupan espacio en memoria pero no están inicializadas con valores específicos.

En el tercer bloque consiste en las subrutinas, cada una de estas funciones tiene su propio propósito específico, ya sea para manejar conversiones, imprimir mensajes, manejar errores o reiniciar variables utilizadas en el programa.

itoa:

Descripción: Convierte un número en una cadena de caracteres (string) para su visualización, especialmente en el editor.

Función: Empieza inicializando una variable para almacenar la cadena resultante, marcando el final de la cadena y añadiendo un punto decimal para diferenciar entre el número de línea y el texto. Luego convierte el número de la base especificada a caracteres ASCII y los almacena en itoaVariable.

itoaHexa:

Descripción: Similar a itoa, pero está preparado para visualizar números en formato hexadecimal.

Función: Inicializa la variable para la cadena en formato hexadecimal y luego llama a startItoa para continuar con la conversión.

itoa2:

Descripción: Otra versión de itoa que convierte un número a cadena de caracteres, específicamente para números decimales.

Función: Realiza la conversión dividiendo el número por 10, transformando cada dígito en un carácter ASCII y almacenándolo en result.

atoi:

Descripción: Convierte una cadena de caracteres numéricos en un número entero.

Función: Recorre la cadena de entrada carácter por carácter, convierte cada carácter de ASCII a su valor numérico y construye el número entero.

restartItoaVariable:

Descripción: Reinicia la variable utilizada para almacenar la cadena en itoa.

Función: Pone todos los bytes de itoaVariable a 0 para prepararla para una nueva conversión.

restartInput:

Descripción: Reinicia la variable de entrada.

Función: Pone todos los bytes de input a 0 para limpiar cualquier entrada previa.

addLines:

Descripción: Añade líneas nuevas al contenido de un archivo.

Función: Inserta el carácter de nueva línea (0xA) en newFileContent hasta que se alcance el número de líneas especificado por rbx.

print:

Descripción: Imprime un mensaje a la salida estándar.

Función: Utiliza la syscall write para imprimir el contenido apuntado por rsi y de longitud rdx.

error:

Descripción: Maneja errores generales.

Función: Imprime un mensaje de error y termina el programa llamando a terminateProgram.

commandError:

Descripción: Maneja errores relacionados con comandos inválidos.

Función: Imprime un mensaje de comando inválido, reinicia la entrada y reinicia el programa desde _start.

printingText:

Descripción: Selecciona y prepara una porción de texto para imprimir.

Función: Copia hasta 80 caracteres de contentNumberLines a textPart para su impresión.

invalidOption:

Descripción: Maneja la selección de opciones inválidas.

Función: Imprime un mensaje de opción inválida y vuelve a leer la opción.

invalidRow:

Descripción: Maneja errores relacionados con filas inválidas.

Punto de entrada principal del programa.

El último bloque, principal _start se detallan las operaciones de lectura, procesamiento y escritura.



Este es el punto de entrada del programa, donde se inicializan las variables y se imprime un mensaje inicial. Luego, se pide al usuario el nombre de un archivo para abrirlo y leer su contenido.

Esta sección recorre el nombre del archivo
ingresado por el usuario y elimina cualquier

Esta sección verifica si hay una opción -h después del nombre del archivo. Si está presente, salta al visualizador hexadecimal, si no, continúa con el visualizador decimal o maneja comandos inválidos.

Convierte el contenido del archivo leído en su representación hexadecimal y lo almacena en `newFileContent`. Luego, salta al final del visualizador hexadecimal.

Agrega números de línea al texto leído del archivo. Convierte los números de línea a ASCII y los inserta en el contenido del archivo.

Cuenta el número de líneas en el archivo leído y almacena este valor en totalRows.

Imprime el contenido del texto con números de línea en pantalla, junto con una interfaz de usuario que indica las acciones posibles.

Lee la opción ingresada por el usuario y llama a la subrutina correspondiente: bajar (goDown), subir (goUp), finalizar (terminateProgram), editar (editText) o buscar diferencias (difference). Maneja opciones inválidas (invalidOption).

Desplaza el texto hacia abajo en la pantalla. Si se intenta desplazar más allá del final del texto, muestra un mensaje de error y vuelve a la selección de opciones.

se intenta desplazar más allá del inicio del texto, muestra un mensaje de error y vuelve a la selección de opciones.

Solicita el nombre de un segundo archivo y compara su contenido con el primero, línea por línea, mostrando las diferencias encontradas.

Permite al usuario editar una línea específica del texto. Solicita el número de línea y el nuevo contenido de la línea, luego reemplaza la línea en `newFileContent`.

Cierra el archivo y termina la ejecución del programa.

finalHexaVisualizer: Cierra el archivo después de la visualización hexadecimal y muestra el contenido en hexadecimal.

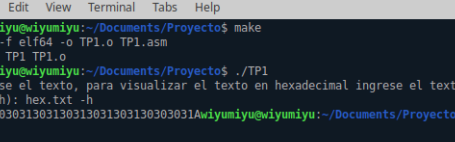
setRowsNumber: Inserta números de línea en el contenido del archivo leído.

iterateRowNumber: Itera sobre el número de línea convertido y lo inserta en el contenido del archivo.

Cada bloque de código está diseñado para manejar una parte específica del flujo del programa, desde la inicialización y entrada del usuario hasta la manipulación y visualización del contenido del archivo.

Este manual le guiará a través de cada una de estas funciones, explicando paso a paso cómo realizar tareas

Para comenzar, asegúrese de tener un entorno Linux con soporte para llamadas al sistema en ensamblador y siga las instrucciones proporcionadas en este manual para instalar y ejecutar la aplicación. Con esta herramienta, podrá manipular archivos de texto de manera eficiente y efectiva, obteniendo una comprensión más profunda de la programación en ensamblador y la interacción directa con el sistema operativo.



The screenshot shows a terminal window titled "Terminal - wiyumiyu@wiyumiyu: ~/Documents/Proyecto". The terminal has a menu bar with "File", "Edit", "View", "Terminal", "Tabs", and "Help". The command prompt is `wiyumiyu@wiyumiyu:~/Documents/Proyecto$`. The user enters `make`, which runs `nasm -f elf64 -o TP1.o TP1.asm` and `ld -o TP1 TP1.o`. Then, the user enters `wiyumiyu@wiyumiyu:~/Documents/Proyecto$ ./TP1`. A prompt appears: "Ingrese el texto, para visualizar el texto en hexadecimal ingrese el texto junto a (-h): hex.txt -h". The user enters `3130303031303130313031303130303031`, followed by `Awiyumiyu@wiyumiyu:~/Documents/Proyecto$`.

```
Terminal - wiyumiyu@wiyumiyu: ~/Documents/Proyecto
File Edit View Terminal Tabs Help
wiyumiyu@wiyumiyu:~/Documents/Proyecto$ make
nasm -f elf64 -o TP1.o TP1.asm
ld -o TP1 TP1.o
wiyumiyu@wiyumiyu:~/Documents/Proyecto$ ./TP1
Ingrese el texto, para visualizar el texto en hexadecimal ingrese el texto junto
a (-h): hex.txt -h
3130303031303130313031303130303031Awiyumiyu@wiyumiyu:~/Documents/Proyecto$
```

Seguidamente el usuario tiene dos maneras de abrir archivos, la primera es abrir un formato binario en hexadecimal y la segunda en formato ASCII.

Para abrir en hexadecimal nada más se digita el nombre del archivo con el comando “-h”; como se observa en la figura 3 el ejemplo. Esta abre el archivo en binario y lo traduce, una vez hecho lo anterior se cierra el programa.

Seguidamente si se quiere manipular de la manera tradicional se digita únicamente el nombre del archivo y de una vez se imprime lo que tiene adentro; pero solo 80 caracteres y para ver más texto se digita la opción “b” de bajar.

```
Terminal - wiyumiyu@wiyumiyu: ~/Documents/Proyecto
File Edit View Terminal Tabs Help
wiyumiyu@wiyumiyu:~/Documents/Proyecto$ ./TP1
Ingrese el texto, para visualizar el texto en hexadecimal ingrese el texto junto a (-h): texto.txt
=====Texto=====
1.Pasarán estos días como pasan
2.todos los días malos de la vida
3.Amainar
=====
Digite la opcion deseada:
a. Editar el texto
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
b
=====Texto=====
án los vientos que te arrasan
4.Se estancará la sangre de tu herida
5.
6.El
=====
Digite la opcion deseada:
a. Editar el texto
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
f
=====Texto=====
alma errante volverá a su nido
7.Lo que ayer se perdió será encontrado
8.E
=====
```

Figura 4: Consola, comando de bajar

Si se requiere de volver a subir nada más se digita la opción “s” y se puede volver a ver lo anterior.

```
Terminal - wiyumiyu@wiyumiyu: ~/Documents/Proyecto
File Edit View Terminal Tabs Help
a. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
b
=====Texto=====
án los vientos que te arrasan
4.Se estancará la sangre de tu herida
5.
6.El
=====
Digite la opcion deseada:
a. Editar el texto
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
b
=====Texto=====
alma errante volverá a su nido
7.Lo que ayer se perdió será encontrado
8.E
=====
Digite la opcion deseada:
a. Editar el texto
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
s
=====Texto=====
án los vientos que te arrasan
4.Se estancará la sangre de tu herida
5.
6.El
=====
```

Figura 4: Consola, comando de subir

Para encontrar las diferencias de archivos se digita la opción “d” seguidamente del otro archivo del cuál se quieren hacer las comparaciones.

```
Terminal - wiyumiyu@wiyumiyu: ~/Documents/Proyecto
File Edit View Terminal Tabs Help
1.en
2.una
3.noche
4.soleada
=====
Digite la opcion deseada:
a. Editar el texto
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
d
Ingrese el texto a comparar:
tetra.txt
=====
Hay una diferencia en la linea: 4
Digite la opcion deseada:
a. Editar el texto
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
f
=====
```

Figura 5: Sección de encontrar las diferencias de archivos

Como se puede observar en la figura 5, la consola menciona la línea que tiene la diferencia.

Y por último si se quiere de editar el archivo como se refleja en la figura 6, nada más se digita “e” y luego se digita la línea a editar, una vez digitado aparece una opción para recibir el texto que el usuario desee y este vuelve a mostrar lo que está en el texto pero con el input del usuario.

```
Terminal - wiyumiyu@wiyumiyu: ~/Documents/Proyecto
File Edit View Terminal Tabs Help
Ingrese el texto, para visualizar el texto en hexadecimal ingrese el texto junto a (-h): carta.txt
=====Texto=====
1.En
2.una
3.noche
4.soleada
=====
Digite la opcion deseada:
a. Editar el texto
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
d
Ingrese el texto a comparar:
tetra.txt
=====
Hay una diferencia en la linea: 4
Digite la opcion deseada:
a. Editar el texto
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
e
4
Ingrese el texto, lluviosa
=====
1.en
2.una
3.noche
4.lluviosa
=====
Digite la opcion deseada:
a. Editar el texto
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
f
=====
wiyumiyu@wiyumiyu:~/Documents/Proyecto$
```

Figura 6: Sección de encontrar las diferencias de archivos

Para salir nada más se digita la opción “f” y se cierra el programa.

IV. RECOMENDACIONES

Una forma sencilla de solucionar un problema que requiera dar al usuario la sensación de navegar por el

texto es manejar el índice del archivo de texto y limitar la cantidad que se imprime en pantalla para darle la opción del usuario qué información quiere ver en todo momento sin tener que leer inputs en tiempo real de teclado y mouse

Para facilitar el manejo de archivos se pueden proporcionar distintos permisos de escritura y lectura, esto permite simplificar el código y evita ciertos errores como intentar abrir un archivo que no existe

Al desarrollar código en ensamblador se debe ser muy consciente de los registros que se están utilizando y los valores que manejan, reutilizar registros que necesarios en otras secciones de código o no manejar correctamente la manipulación de datos al hacer syscalls puede llevar a errores en el programa

V. CONCLUSIONES

Con base en los objetivos establecidos, se pueden derivar las siguientes conclusiones:

Completa Implementación Funcional: El proyecto logró cumplir con éxito los objetivos establecidos al desarrollar un editor, visualizador y comparador de archivos totalmente funcional en ensamblador x8086 ASM. Se logró implementar las funciones básicas de edición, así como algoritmos eficientes para la visualización y comparación de archivos de texto.

Optimización del Rendimiento: La elección de ensamblador x8086 ASM permitió optimizar el rendimiento del software al máximo, aprovechando el control de bajo nivel sobre el hardware. Esto resultó en un editor de archivos eficiente y rápido, proporcionando una experiencia fluida para los usuarios.

Planificación Estructurada: La planificación cuidadosa del proyecto, incluyendo el diseño de diagramas de flujo y diagramas de bloques funcionales, fue fundamental para visualizar la estructura y el funcionamiento del software antes de su implementación. Esto garantizó una codificación más eficiente y una mayor comprensión del proyecto en su conjunto.

Contribución Educativa: El proyecto no solo cumplió con sus objetivos funcionales, sino que también sirvió como un valioso recurso educativo en el ámbito de la programación a bajo nivel. Proporcionó una experiencia práctica en el manejo directo de hardware y en la programación en ensamblador, lo que contribuyó al desarrollo de habilidades técnicas avanzadas.

En resumen, el proyecto demostró ser una herramienta robusta y eficiente para la edición, visualización y comparación de archivos, destacando por su implementación en ensamblador x8086 ASM y por su

enfoque estructurado y educativo en el desarrollo de software a bajo nivel.

VI. LITERATURA CONSULTADA

Arm. (2023, July 18). ARM Cortex-R Series Programmer's Guide. <https://developer.arm.com/documentation/den0042/a/Introduction-to-Assembly-Language>

Writing ARM Assembly (Part 1). (n.d.). Azeria-Labs. <https://azeria-labs.com/writing-arm-assembly-part-1/>

EDITOR

Macros

Imprimir

Datos

Data

msgError

msgInvalidCommand

file

digits

msgRow

msgText

msgInvalidRow

nombre

Bss

fileName

input

fileDescriptor

fileContent

newFileContent

contentNumberLines

borrar

itoaVariable

countDigits

rowNumberInput

totalRows

inputText

Funciones y Subrutinas

itoa

itoaHexa

itoa2

atoi

restartItoaVariable

invalidOption

invalidRow

restartInput

addLines

print

error

commandError

printingText

Start

_start

deleteEndLine

checkHexadecimalVisualizer

hexaVisualizer

decimalVisualizer

rowCounter

printText

readOption

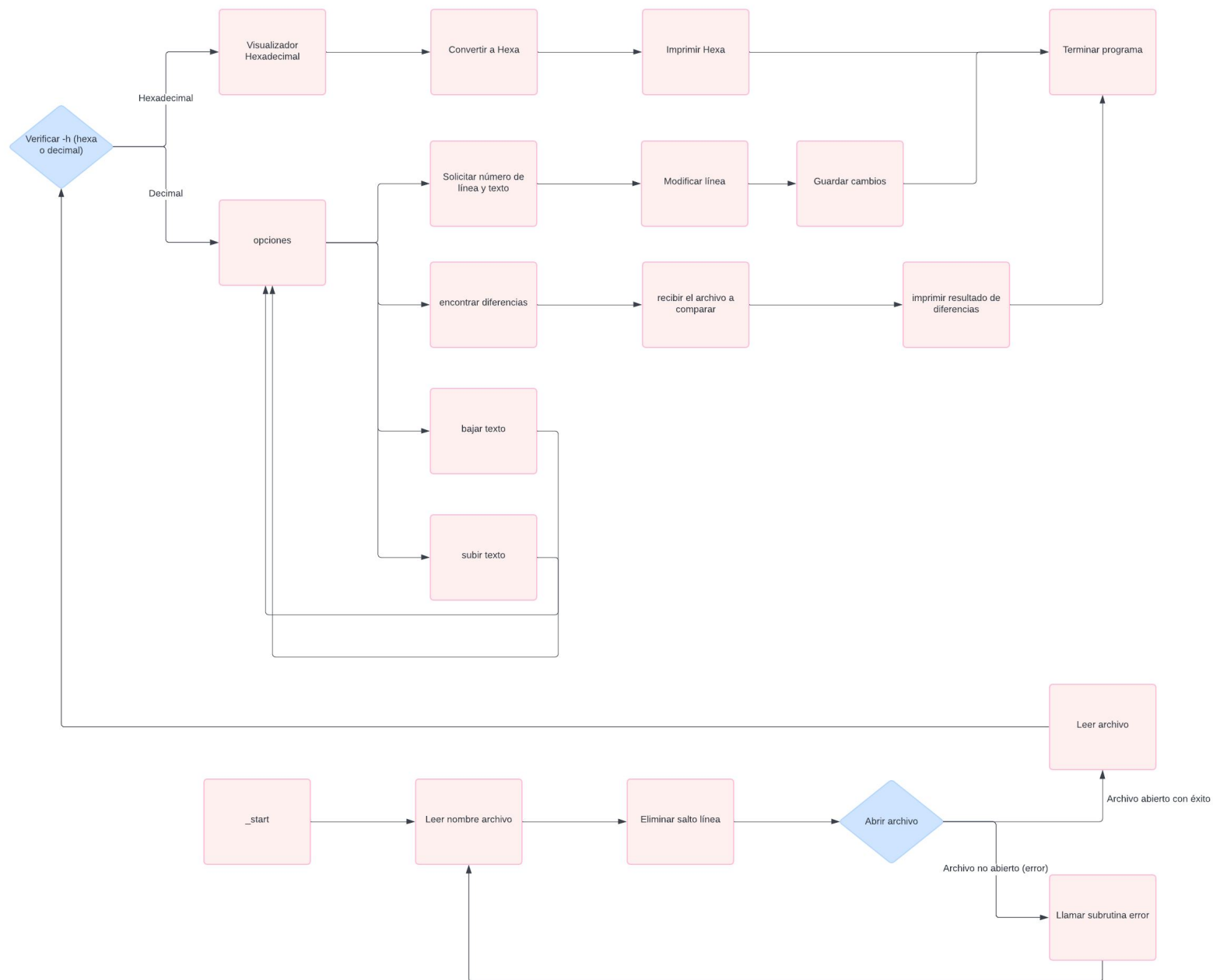
goDown

goUp

difference

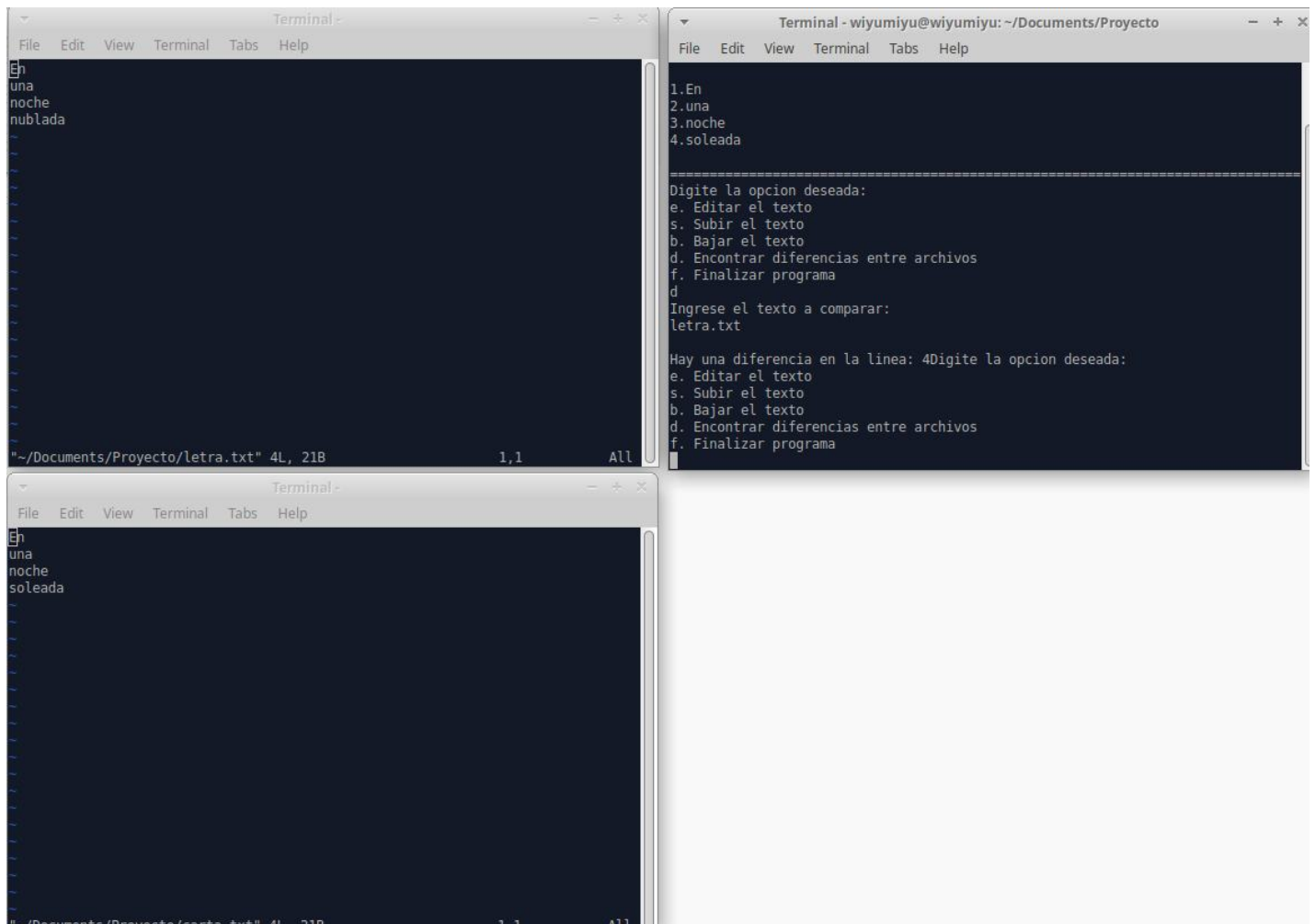
editText

terminateProgram



```
Terminal - wiyumiyu@wiyumiyu: ~/Documents/Proyecto
File Edit View Terminal Tabs Help

wiyumiyu@wiyumiyu:~/Documents/Proyecto$ make
nasm -f elf64 -o TP1.o TP1.asm
ld -o TP1 TP1.o
wiyumiyu@wiyumiyu:~/Documents/Proyecto$ ./TP1
Ingrese el texto, para visualizar el texto en hexadecimal ingrese el texto junto
a (-h): hex.txt -h
313030303130313031303130313031A wiyumiyu@wiyumiyu:~/Documents/Proyecto$
```



```
Terminal - wiyumiyu@wiyumiyu: ~/Documents/Proyecto
File Edit View Terminal Tabs Help
Ingrese el texto, para visualizar el texto en hexadecimal ingrese el texto junto
a (-h): carta.txt
=====Texto=====
1.En
2.una
3.noche
4.soleada
=====
Digite la opcion deseada:
e. Editar el texto
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
d
Ingrese el texto a comparar:
letra.txt
Hay una diferencia en la linea: 4
Digite la opcion deseada:
e. Editar el texto
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
e
4
Ingrese el texto, lluviosa
En
una
noche
lluviosa
Digite la opcion deseada:
e. Editar el texto
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
f
wiyumiyu@wiyumiyu:~/Documents/Proyecto$
```

```
Terminal - wiyumiyu@wiyumiyu: ~/Documents/Proyecto
File Edit View Terminal Tabs Help
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
b
=====Texto=====
án los vientos que te arrasan
4.Se estancará la sangre de tu herida
5.
6.El
=====
Digite la opcion deseada:
e. Editar el texto
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
b
=====Texto=====
alma errante volverá a su nido
7.Lo que ayer se perdió será encontrado
8.E
=====
Digite la opcion deseada:
e. Editar el texto
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
s
=====Texto=====
án los vientos que te arrasan
4.Se estancará la sangre de tu herida
5.
6.El
```

```
Terminal - wiyumiyu@wiyumiyu: ~/Documents/Proyecto
File Edit View Terminal Tabs Help
wiyumiyu@wiyumiyu:~/Documents/Proyecto$ ./TP1
Ingrese el texto, para visualizar el texto en hexadecimal ingrese el texto junto
a (-h): texto.txt
=====Texto=====
1.Pasarán estos días como pasan
2.todos los días malos de la vida
3.Amainar
=====
Digite la opcion deseada:
e. Editar el texto
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
o
=====Texto=====
án los vientos que te arrasan
4.Se estancará la sangre de tu herida
5.
6.El
=====
Digite la opcion deseada:
e. Editar el texto
s. Subir el texto
b. Bajar el texto
d. Encontrar diferencias entre archivos
f. Finalizar programa
o
=====Texto=====
alma errante volverá a su nido
7.Lo que ayer se perdió será encontrado
8.E
=====
```